

K.RAMAKRISHNAN
COLLEGE OF ENGINEERING
An Autonomous Institution
Permanently Affiliated to Anna University Chennai, Approved by AICTE New Delhi,
ISO 9001:2015, 14001:2015 certified institution, Accredited by NBA and with A grade by NAAC
Samayapuram, Tiruchirappalli – 621 112, Tamilnadu, India.



Stack Calculator Evaluating Arithmetic Expressions

PRESENTED BY

NAME : SRUTI LAVANYA S

REGISTER NO : 8115U25AM056

SUPERVISOR

Mrs.P.GEETHA

ASSISTANT PROFESSOR / AI&DS

PRESENTATION OVERVIEW

- Project Introduction
- Problem Statement
- What is stack?
- Types of expressions
- Architecture Diagram
- Operator used
- Modules and purpose
- Output Screenshots
- Future scope
- Conclusion
- References

OBJECTIVES

- The main objective of this project is to evaluate arithmetic expressions using the stack data structure,It helps convert infix expressions (like A + B) into postfix form (A B +) and then calculate the result easily by following the LIFO principle of the stack.

PROJECT INTRODUCTION

- "This project uses the Stack data structure to convert and evaluate arithmetic expressions efficiently while maintaining operator precedence and correct order of execution."

PROBLEM STATEMENT

- Parentheses increase difficulty
- Operator precedence handling is complex
- Stack simplifies evaluation process

WHAT IS STACK?

- Linear Data Structure
- Last In First Out (LIFO)
- Operations: Push, Pop, Peek, IsEmpty
- Example: Stack of plates

TYPES OF EXPRESSION

- Infix → A + B
- Postfix → A B +
- Prefix → + A B
- Project uses Infix to Postfix conversion

OPERATORS USED

- +, - → Level 1
- *, / → Level 2
- Higher precedence evaluated first

DIAGRAM

Infix Expression (5 + 3) * (8 - 2)

Postfix Expression 5 3 + 8 2 - *

Above Postfix Expression can be evaluated by using Stack Data Structure as follows...

Reading Symbol	Stack Operations	Evaluated Part of Expression
Initially	Stack is Empty	Nothing
5	push(5)	Nothing
3	push(3)	Nothing

+	value1 = pop()
	value2 = pop()
	result = value2 + value1
	push(result)
		value1 = pop(); // 3
		value2 = pop(); // 5
		result = 5 + 3; // 8
		Push(8)
		(5 + 3)

8	push(8)	(5 + 3)
2	push(2)	(5 + 3)

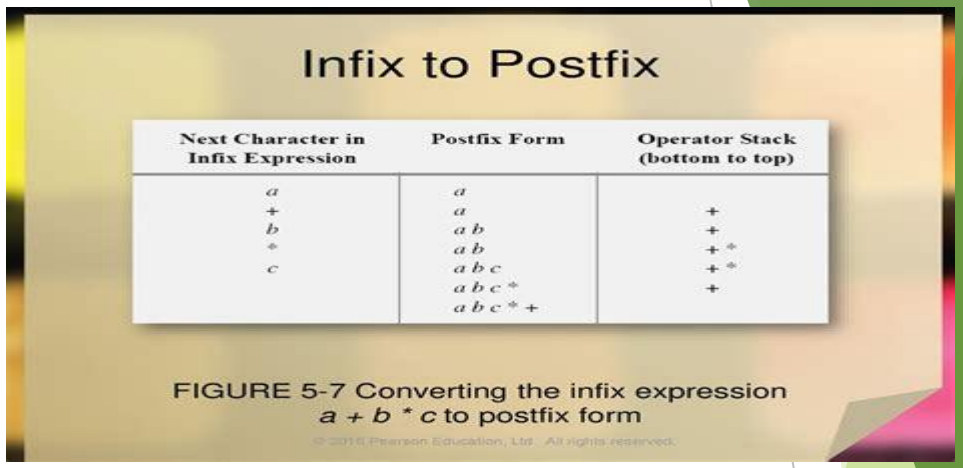
-	value1 = pop()
	value2 = pop()
	result = value2 - value1
	push(result)
		value1 = pop(); // 2
		value2 = pop(); // 8
		result = 8 - 2; // 6
		Push(6)
		(8 - 2)
		(5 + 3) , (8 - 2)

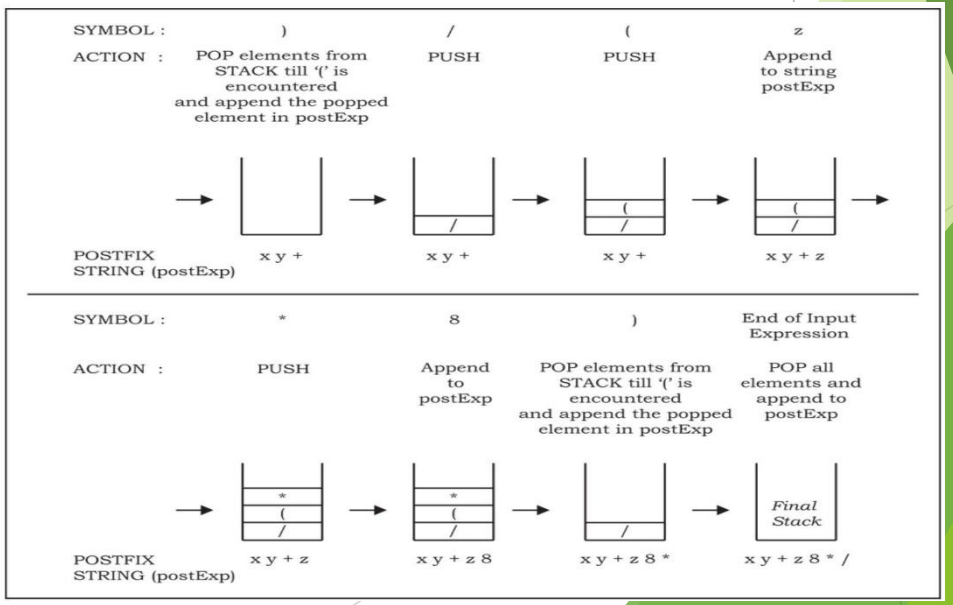
*	value1 = pop()
	value2 = pop()
	result = value2 * value1
	push(result)
		value1 = pop(); // 6
		value2 = pop(); // 8
		result = 8 * 6; // 48
		Push(48)
		(6 * 8)
		(5 + 3) * (8 - 2)

$ End of Expression	result = pop()	Display (result) 48 As final result

Infix Expression (5 + 3) * (8 - 2) = 48

Postfix Expression 5 3 + 8 2 - * value is 48





MODULES USED AND PURPOSE

MODULE	PURPOSE
INPUT / UI	GETS EXPRESSION FROM USER
TOKENIZER	BREAKS EXPRESSION INTO TOKENS
INFIX → POSTFIX CONVERTER	CONVERTS NOTATION USING STACK
POSTFIX EVALUATOR	COMPUTES NUMERIC RESULT USING STACK
ERROR HANDLER	ENSURES VALID INPUT AND CATCHES ERRORS
OUTPUT	DISPLAYS RESULT
MAIN CONTROLLER	ORCHESTRATES THE FLOW

OUTPUT

main.c

Share | Run

```
            int a = st[t--];

            switch (ch)
            {
                case '+': st[++t] = a + b; break;
                case '-': st[++t] = a - b; break;
                case '*': st[++t] = a * b; break;
                case '/': st[++t] = a / b; break;
            }
        }
    }
    return st[t];
}

int main()
{
    char infix[100], postfix[100];

    printf("Enter Infix Expression: ");
    scanf("%s", infix);
```

Output | Clear

```
Enter Infix Expression: 53+90*7
Postfix Expression: 53907*+
Evaluated Result: 9


=== Code Execution Successful ===
```

FUTURE SCOPE

- Support for advanced mathematical operations.
- Graphical User Interface (GUI) enhancement.
- Support for variables and expression history.
- Error handling and validation improvements.
- Visualization of computation process.

CONCLUSION

This project shows that stacks are very useful in solving real-world problems like expression evaluation.It provides a simple, reliable, and efficient way to handle operators and operands.The project helps understand how data structures and algorithms work together in real applications like calculators and compilers.

REFERENCES

1. **Infix, Prefix, and Postfix Expressions (Open Book Project)** – A digital textbook section on evaluating postfix expressions with clear stack logic, similar to project requirements.
2. **Shunting Yard Algorithm (Wikipedia)** – Classic algorithm used in many textbooks to explain infix → postfix conversion using stacks.
3. **Expression Evaluation Using Stack (Naukri Code360)** – Includes infix, postfix, and prefix conversion and evaluation using stacks with algorithm and examples.
   https://www.naukri.com/code360/library/expression-evaluation-using-stack
4. **Stack Visualization Tool** – An interactive tool to visualize the stack steps for conversion and evaluation, useful as a learning reference.
   https://www.stackvisualizer.online/
5. **Case Study: Infix to Postfix (Lecture PDF)** – A simple educational case study showing infix to postfix evaluation with example steps.
   https://students-hub.com/en/view-resource/26c7e77b-6479-4c12-b392-e70e5d774973/Lecture_6_Stacks_part2.pdf
6. **Evaluation of Postfix Expression (GeeksforGeeks)** – Shows how a postfix expression can be evaluated using a stack with examples and code.
   https://www.geeksforgeeks.org/dsa/evaluation-of-postfix-expression/


THANK
YOU